BZ205 Konu Anlatımı

Ek – B+ Ağaçları ve Graf Arama

Hocanın ders videoları, slaytları ve sınav soruları temel alınmıştır

27 Ekim 2026

9 Ek: B+ Ağaçları ve Graf Arama (bazı yıllarda vizeye girdi)

2020 ve 2024 vizelerinde B+, 2024 vizesinde ve 2024–25 mazeret sınavında graf (BFS/sonrakök) soruldu. Hoca konuları hızlı işlerse vizeye girer.

9.1 B+ ağacı

- Çocuk sayısı 3 ve daha fazla olan çok yollu dengeli ağaç (*M-ary*); çocuk sayısı arttıkça yükseklik $\log_M n$'e düşer, disk erişimi azalır.
- **Bütün veriler yapraklardadır**, yapraklar aynı seviyededir; iç düğümler sadece yol gösteren *anahtarları* tutar. Bir iç düğümdeki i. anahtar, $i+1$. alt ağacın en küçük verisidir (*"sağdakinin minimumu yukarı anahtar olarak kopyalanır"*).
- **Hocanın formülleri** (M: iç düğüm çocuk sayısı, L: yaprak parametresi; aksi söylenmezse $L = M$):

	En fazla	En az
İç düğüm anahtar	$M-1$	$\lceil M/2 \rceil - 1$
Yaprak veri	$L-1$	$\lceil L/2 \rceil - 1$
Kök	$M-1$	1 (kökün "süperliği")

  Soru dilinden çeviri: "iç düğüm çocuk sayısı 4" $\Rightarrow M = 4$; "iç düğüm anahtar sayısı 3" $\Rightarrow M = 4$; "yapraklarda tutulan maksimum veri 4" $\Rightarrow L = 5$ (2025 final için WhatsApp'ta da M=4, L=5 sonucuna varılmış).
- **Ekleme:** yaprağa sıralı eklenir. Yaprak taşarsa (L veri): sola $\lfloor L/2 \rfloor$ en küçük, sağa kalanlar; *sağdakinin en küçüğü yukarı kopyalanır*. İç düğüm taşarsa (M anahtar): sola $\lceil M/2 \rceil - 1$ anahtar, $\lceil M/2 \rceil$. anahtar yukarı *taşınır*, kalanlar sağa. Kök bölünürse yeni kök oluşur.
- **Silme:** veri yapraktan silinir. Yaprak minimumun altına düşerse *sağ kardeşiyle* (en sağdaki yaprak ise sol kardeşiyle) **birleştir–böl** (veriler iki yaprağa yeniden paylaştırılır); bu da minimumu sağlayamıyorsa iki yaprak **birleştirilir** ve üstteki anahtar silinir (iç düğüm de aynı mantıkla düzeltilir). Anahtarlar her zaman "sağ alt ağacın en küçüğü" olacak şekilde güncellenir. *"B şıkkını yanlış yaparsanız C şıkkı da otomatikman yanlış olacak."*

Örnek: $M = L = 4$ ile K, L, M, N, O, P, Q, R, S, T ekle; N ve O sil

▷ Bu bölümün adım adım oynatılabilen etkileşimli animasyonu HTML sürümündedir.

Parametreler: iç düğüm çocuk sayısı $M = 4 \Rightarrow$ iç düğümde en fazla $M - 1 = 3$, en az $\lceil M/2 \rceil - 1 = 1$ anahtar; $L = 4 \Rightarrow$ yaprakta en fazla $L - 1 = 3$, en az $\lceil L/2 \rceil - 1 = 1$ veri.

Başlangıç ağacı



K ekle: K yaprağa eklendi (yaprakta boş yer vardı).



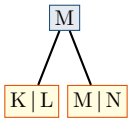
L ekle: L yaprağa eklendi (yaprakta boş yer vardı).



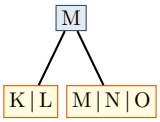
M ekle: M yaprağa eklendi (yaprakta boş yer vardı).



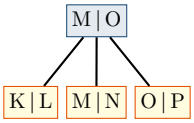
N ekle: N eklenince yaprakta 4 veri oldu (maks. 3) ⇒ yaprak bölünür: sola $\lfloor 4/2 \rfloor$=2 en küçük (K, L), sağa 2 (M, N); sağdakinin en küçüğü M yukarı anahtar olarak KOPYALANIR. Kök bölündüğü için yeni kök oluştu: [M].



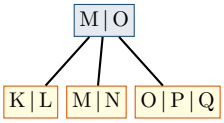
O ekle: O yaprağa eklendi (yaprakta boş yer vardı).



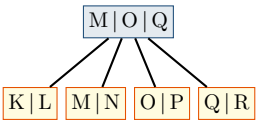
P ekle: P eklenince yaprakta 4 veri oldu (maks. 3) ⇒ yaprak bölünür: sola $\lfloor 4/2 \rfloor$=2 en küçük (M, N), sağa 2 (O, P); sağdakinin en küçüğü O yukarı anahtar olarak KOPYALANIR.



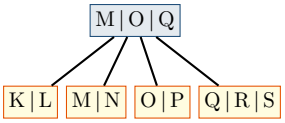
Q ekle: Q yaprağa eklendi (yaprakta boş yer vardı).



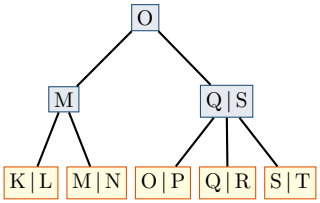
R ekle: R eklenince yaprakta 4 veri oldu (maks. 3) ⇒ yaprak bölünür: sola $\lfloor 4/2 \rfloor$=2 en küçük (O, P), sağa 2 (Q, R); sağdakinin en küçüğü Q yukarı anahtar olarak KOPYALANIR.



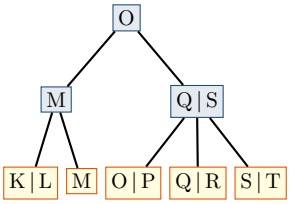
S ekle: S yaprağa eklendi (yaprakta boş yer vardı).



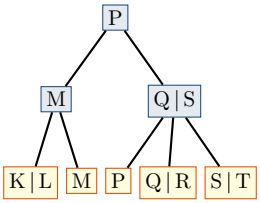
T ekle: T eklenince yaprakta 4 veri oldu (maks. 3) ⇒ yaprak bölünür: sola $\lfloor 4/2 \rfloor = 2$ en küçük (Q, R), sağa 2 (S, T); sağdakinin en küçüğü S yukarı anahtar olarak KOPYALANIR. İç düğümde 4 anahtar oldu (maks. 3) ⇒ iç düğüm bölünür: sola $\lceil 4/2 \rceil - 1 = 1$ anahtar (M), O yukarı TAŞINIR, sağa Q, S. Kök bölündüğü için yeni kök oluştu: [O].



N sil: N yapraktan silindi. Minimum kural sağlanıyor; gerekirse üst anahtarlar sağdakinin en küçüğü ile güncellenir.



O sil: O yapraktan silindi. Minimum kural sağlanıyor; gerekirse üst anahtarlar sağdakinin en küçüğü ile güncellenir.



Zaman ve alan karmaşıklığı

İşlem	En iyi	En kötü	Ek alan	Not
Arama	$\Omega(\log_M n)$	$O(\log_M n)$	$O(1)$	kökten yaprağa bütün yollar eşit uzunlukta (yapraklar aynı seviyede)
Ekleme	$\Omega(\log_M n)$	$O(\log_M n)$	$O(1)$	bölünme köke kadar yayılabilir; her seviyede sabit iş
Silme	$\Omega(\log_M n)$	$O(\log_M n)$	$O(1)$	birleştir–böl / birleştirme köke kadar yayılabilir
Sıralı gezme	$\Theta(n)$	$\Theta(n)$	$O(1)$	yapraklar soldan sağa okunur

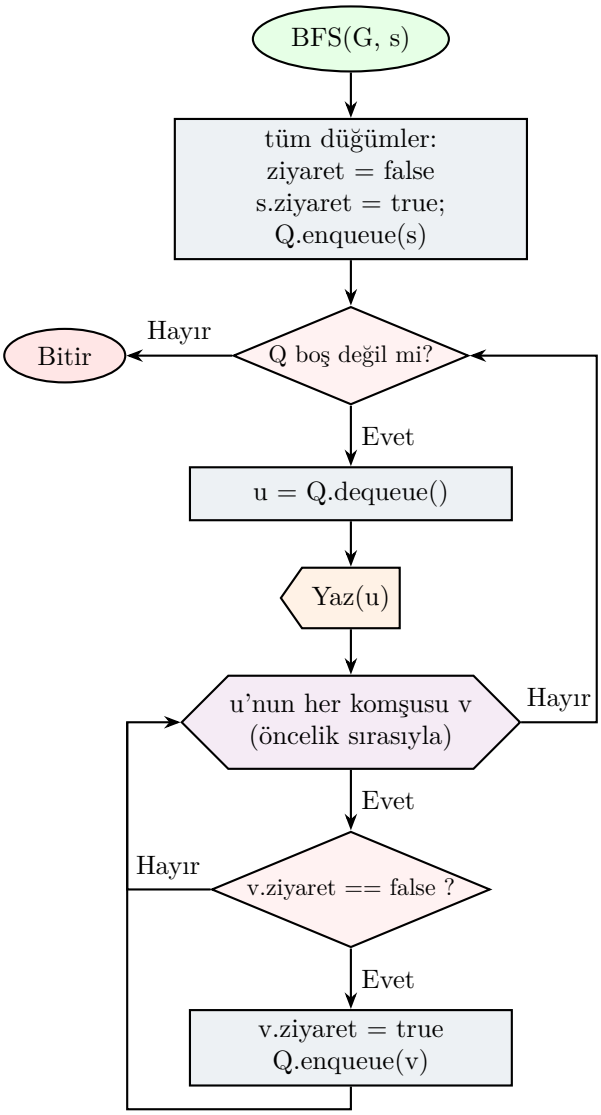
9.2 Graflar: BFS, DFS ve sonrakök

- Gösterim: komşuluk matrisi (n^2 yer) veya komşuluk listesi (seyrek graflarda tercih edilir).
- **Genişlik öncelikli arama (BFS):** başlangıç kuyruğa konur; kuyruktan çıkan düğümün ziyaret edilmemiş komşuları (soruda verilen alfabetik önceliğe göre) kuyruğa eklenir. Ağırlıksız grafta en kısa yolları (kenar sayısı) verir. $O(V + E)$.
- **Derinlik öncelikli arama (DFS):** yığın / özyineleme; bir komşuya gidip olabildiğince derine inilir. Öncekök sırası = ilk ziyaret sırası; **sonrakök sırası = bir düğümün bütün komşuları bitince yazılması**.
- **Öncelik notuna dikkat:** "alfabetik olarak önde olanı önce" veya "sonda olanı önce" (2024 vize, mazeret, 2020 büt) – komşuluk listesini buna göre sıralayın.

Akış diyagramı: genişlik öncelikli arama (BFS) – kuyruk ile

```
BFS(G, s)
  her u icin u.ziyaret = false
  s.ziyaret = true;  Q.enqueue(s)
  while (!Q.isEmpty())
     u = Q.dequeue();  Yaz(u)
     for (her komsu v, oncelik sirasiyla)
        if (v.ziyaret == false)
           v.ziyaret = true
           Q.enqueue(v)
```

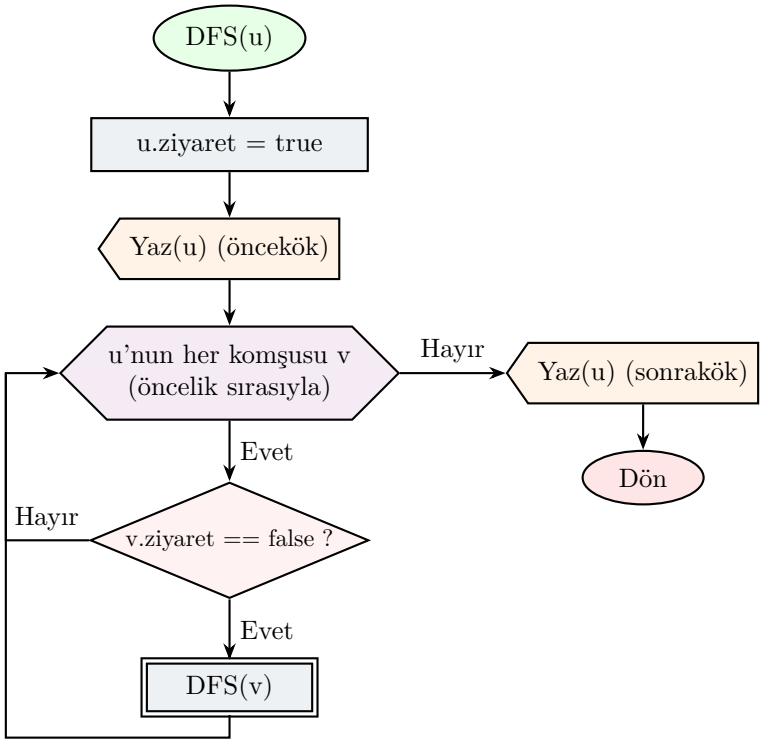
Komşu önceliği soruda verilir (alfabetik önde/sonda). 2025–26 finalinde BFS akış diyagramı ve karmaşıklığı soruldu.


BFS(G, s)
tüm düğümler:
ziyaret = false
s.ziyaret = true;
Q.enqueue(s)
Q boş değil mi?
Hayır
Bitir
Evet
u = Q.dequeue()
Yaz(u)
u'nun her komşusu v
(öncelik sırasıyla)
Hayır
Evet
v.ziyaret == false ?
Hayır
Evet
v.ziyaret = true
Q.enqueue(v)

Akış diyagramı: derinlik öncelikli arama (DFS) ve sonrakök sırası

```
DFS(u)
  u.ziyaret = true
  Yaz(u)            // oncekok sirasi
  for (her komsu v, oncelik sirasiyla)
     if (v.ziyaret == false)
        DFS(v)
  Yaz(u)            // sonrakok sirasi
```

Özyineleme arka planda yığın kullanır. "Düğümleri sonrakök mantığına göre sıralayınız" sorusunun cevabı ikinci `Yaz` satırının ürettiği sıradır.



Zaman ve alan karmaşıklığı

İşlem	En iyi	En kötü	Ek alan	Not
BFS (komşuluk listesi)	$\Theta(V+E)$	$\Theta(V+E)$	$O(V)$	her düğüm bir kez kuyruğa girer, her kenara bir kez bakılır
DFS / sonrakök	$\Theta(V+E)$	$\Theta(V+E)$	$O(V)$	özyineleme (yığın) derinliği en fazla V
Komşuluk matrisiyle	$\Theta(V^2)$	$\Theta(V^2)$	$O(V^2)$	her düğüm için bütün satır taranır

Örnek (ders slaytındaki graf, A'dan, alfabetik önde öncelikli)

▷ Bu bölümün adım adım oynatılabilen etkileşimli animasyonu HTML sürümündedir.

Komşuluk listesi (komşular alfabetik olarak ÖNDE olan önce sıralanmış):

A: C, D, E

B: –

C: A, E

D: A, E

E: A, C, D

Genişlik öncelikli arama (BFS) — kuyruk kullanılır:

Çıkan	Komşular	Kuyruğa eklenen	Kuyruk
A	C, D, E	C, D, E	C, D, E
C	A, E	–	D, E
D	A, E	–	E
E	A, C, D	–	boş

BFS ziyaret sırası: A, C, D, E

Uzaklıklar (kenar sayısı): A=0; C=1; D=1; E=1

Derinlik öncelikli gezme ve sonrakök (postorder) — bir düğümün bütün (ziyaret edilmemiş) komşuları bitince düğüm listeye yazılır (yığın mantığı):

A ziyaret edildi, yığın: [A]

C ziyaret edildi, yığın: [A, C]

E ziyaret edildi, yığın: [A, C, E]

D ziyaret edildi, yığın: [A, C, E, D]

D **bitti** ⇒ listeye yaz; yığın: [A, C, E]

E **bitti** ⇒ listeye yaz; yığın: [A, C]

C **bitti** ⇒ listeye yaz; yığın: [A]

A **bitti** ⇒ listeye yaz; yığın: []

DFS (öncekök) ziyaret sırası: A, C, E, D

Sonrakök (postorder) sırası: D, E, C, A

Not: B düğüm(ler)ine A'den yol yoktur, bu yüzden listeye girmez.